Manual del Programador

Proyecto: Suite de Taller de Repaso e Integración (Actividad 6)
Materia: Laboratorio de Programación (LPR) — 5° Año
Institución: E.E.S.T. N° 20 "Bangtan Sonyeondan" — Vicente López
Estudiante: Alexander Veit
Archivo principal: `src/main.cpp`

---

1. Introducción General

El presente documento técnico constituye el manual del programador para el código correspondiente a la **Actividad 6: Suite de Taller de Repaso e Integración**. El objetivo general de esta aplicación es integrar y poner a prueba conceptos clave del lenguaje C++ y de la gestión de recursos a bajo nivel, estructurados en tres retos principales:

1. **Recursividad:** Procesamiento iterativo implícito para el cálculo de suma acumulada.
2. **Arreglos y Búsqueda Lineal:** Búsqueda en estructuras contiguas de memoria e inspección de direcciones RAM reales.
3. **Punteros e Intercambio de Memoria:** Manipulación directa de celdas mediante referencias y desreferenciación.

---

2. Requisitos del Sistema y Compilación

Requisitos Técnicos

- **Compilador C++:** `g++` (GCC) v5.0 o superior, o cualquier

compilador compatible con C++11 o superior.
- **Entorno de ejecución:** Consola de comandos Linux, macOS o Windows (PowerShell/CMD).

Comandos de Compilación y Ejecución

Para compilar el proyecto manualmente desde la raíz del código fuente:

```
# Compilacion con standard C++11
g++ -std=c++11 src/main.cpp -o suite_integracion

# Ejecucion del ejecutable
./suite_integracion
```

---

3. Arquitectura y Estructura del Código

El archivo `src/main.cpp` está estructurado modularmente con prototipos de función al inicio, la función principal `main()`, y la implementación de las funciones de apoyo al final.

Flujo Principal (`main`)

1. **Reto 1:** Invoca a la función recursiva `sumaRecursiva(5)` y muestra el resultado en consola.
2. **Reto 2:** Inicializa un arreglo estático de 10 enteros (`vectorDatos`), ejecuta un bucle de búsqueda secuencial $O(n)$ del valor `18`, e imprime tanto el índice como la dirección de memoria hexadecimal en RAM.
3. **Reto 3:** Asigna valores a dos variables enteras (`x` e `y`), invoca la función `intercambiarValores` enviando sus direcciones de memoria (`&x`, `&y`) y verifica los valores actualizados.

---

4. Documentación API y Funciones Modulares

4.1 `sumaRecursiva`

Suma todos los enteros positivos de 1 hasta n de forma recursiva.

```
int sumaRecursiva(int n);
```

- **Parámetros:**
  - `n` (`int`): Límite superior para el cálculo de la suma acumulada.
- **Retorno:**
  - `int`: Suma total acumulada ($\sum_{i=1}^{n} i$). Retorna `0` si $n \le 0$.
- **Detalles Técnicos:**
  - **Caso Base:** `n <= 0` retorna `0`. Esto previene el desbordamiento de pila (*Stack Overflow*).
  - **Complejidad Temporal:** $O(n)$
  - **Complejidad Espacial:** $O(n)$ debido a las llamadas en el *call stack*.

4.2 `intercambiarValores`

Realiza un intercambio físico de valores en memoria entre dos celdas enteras utilizando punteros (*Pass-by-Pointer*).

```
void intercambiarValores(int* ptrA, int* ptrB);
```

- **Parámetros:**
  - `ptrA` (`int*`): Puntero apuntando a la dirección de memoria de la primera variable.
  - `ptrB` (`int*`): Puntero apuntando a la dirección de memoria de la segunda variable.

- **Retorno:** `void` (Modificación *in-place* por efecto secundario).
- **Detalles Técnicos:**
  - Utiliza una variable auxiliar `temporal` para preservar el valor de `*ptrA` antes de la sobrescritura.
  - Accede al contenido directo de las direcciones utilizando el operador de desreferenciación (`*`).

---

5. Salida Esperada en Consola

Al ejecutar la aplicación, el resultado presentado en la terminal sigue este formato:

```
=======================================================
  TALLER INTEGRADOR REPASO - ESTUDIANTE: Alexander Veit
=======================================================

=== RETO 1: SUMA RECURSIVA ===
Suma acumulada de 1 hasta 5: 15

=== RETO 2: BUSQUEDA EN MEMORIA CONTIGUA ===
[SISTEMA] Valor 18 hallado en el indice [6]
          Direccion RAM Hexadecimal: 0x7ff7bfe0b1e8

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos  -> X: 100 | Y: 500
Valores actuales -> X: 500 | Y: 100
=======================================================
```

> **Nota:** La dirección hexadecimal (`0x7ff...`) variará según la asignación de memoria dinámica/estática en el momento de la ejecución